Kotlin Backend Developer.Professional

otus.ru

REC **Проверить, идет ли запись**

Меня хорошо видно && слышно?

Репозитарий с Cassandra

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
TG OTUS Kotlin-2025-05

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Программа курса

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multi-platform	Interop	Gradle
Модуль 3: Подготовка к разработке	Выбор проекта	Маркетинг аналитика	Security	Архитек-тура	DevOps	
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Работа с БД	Репози-торий	Конкурен-ция	Postgres	Cassandra	Gremlin
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде	Обзор экосист.	
Модуль 9: Проектная работа	Защита					

Окатов Сергей
автор программы, руководитель курса

- кандидат физ.-мат. наук
- 15 лет научной работы в университетах РФ, Европы, США
- 10 лет комм. разработки в стартапах и корпорациях (Сбер, Ланит) в качестве руководителя разработки, архитектора
- Специализация
  - Котлин
  - Большие данные
  - Распределенные системы
  - Архитектура

План полета

- Особенности Cassandra
- Создание репозитория
- Тестирование
- Интеграция в приложение
- Рефлексия

Цели вебинара

После занятия вы сможете

1.	Узнать об особенностях устройства Cassandra и ее применении для хранения больших данных
2.	Создать репозитарий с использованием Cassandra
3.	Интегрировать репозитарий в приложении совместно с тестовыми БД

Смысл

Зачем вам это уметь

1.	Cassandra - яркий представитель BigData.
2.	BigData позволяет реализовывать более мощные и быстрые проекты чем SQL
3.	У Cassandra есть хитрости, которые позволяют ускорить работу базы

Особенности Cassandra

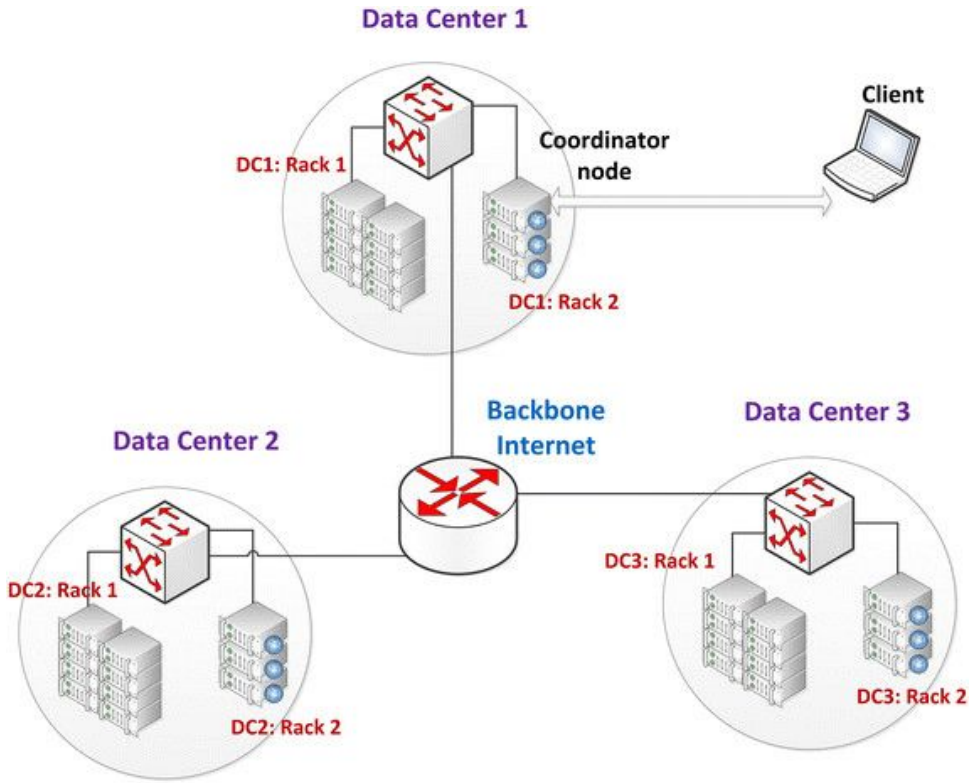
Общие сведения о Cassandra

- Распределенная база данных
- В том числе гео-распределение по датацентрам
- Простое масштабирование
- Неограниченное количество данных
- Производительность пропорциональна числу узлов
- Беспрецедентная живучесть
- Колоночная база данных
- Нет реляционной модели (CQL, не SQL)
- Нет операции OR в WHERE
- Все запросы асинхронные
- Батчи применяются только для атомарных операций

Устройство Cassandra

Элементы:

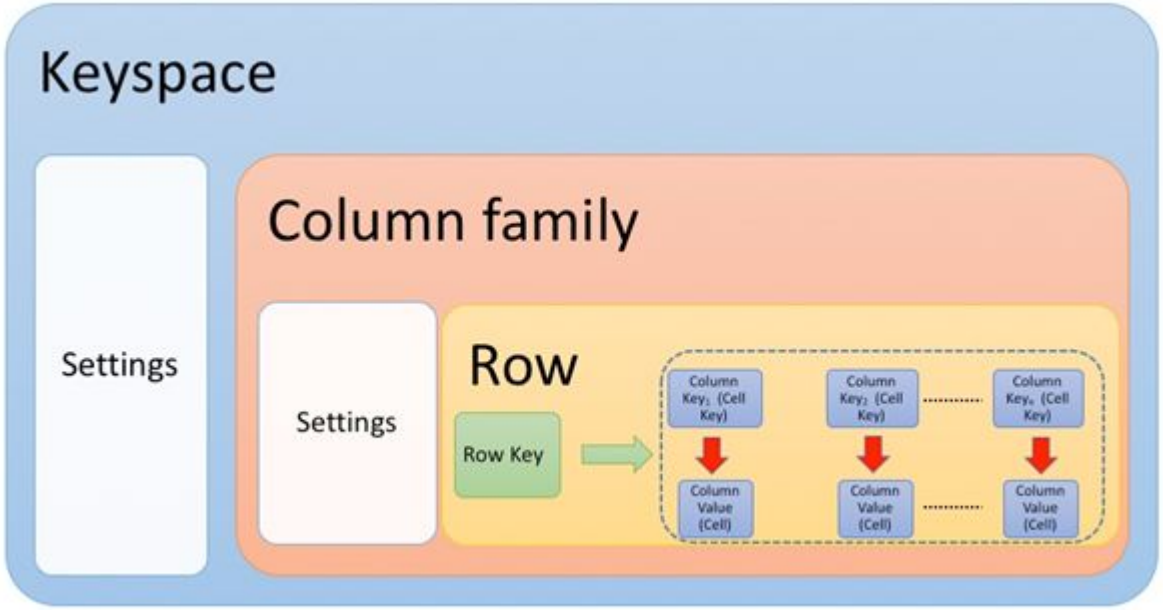
- Кластер - семейство дата-центров
- Дата-центр - семейство узлов в пределах одной сети
- Узлы - сервера с установленной Кассандрой



Устройство Cassandra

Элементы:

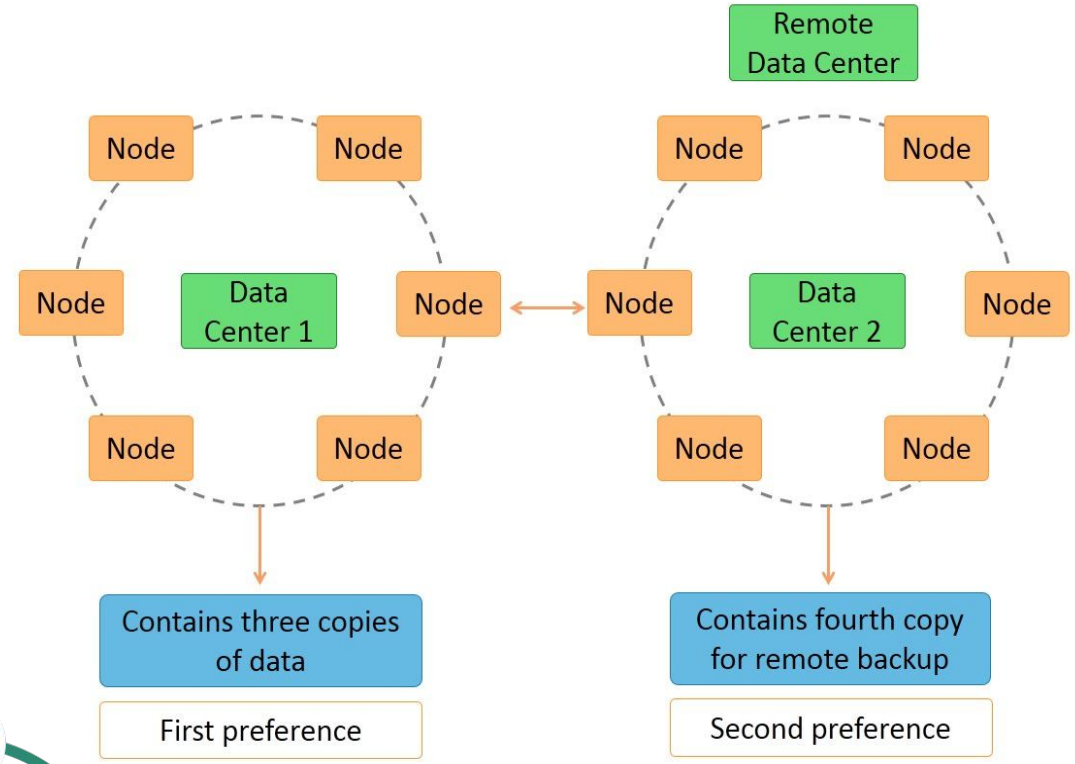
- Пространства ключей (keyspace) - “база данных”, может содержать разные таблицы.
- Таблица (семейство колонок, column family) - содержит строки с данными.
- Строка (row) – набор ячеек
- Токен (token, row key) - уникальный идентификатор строки

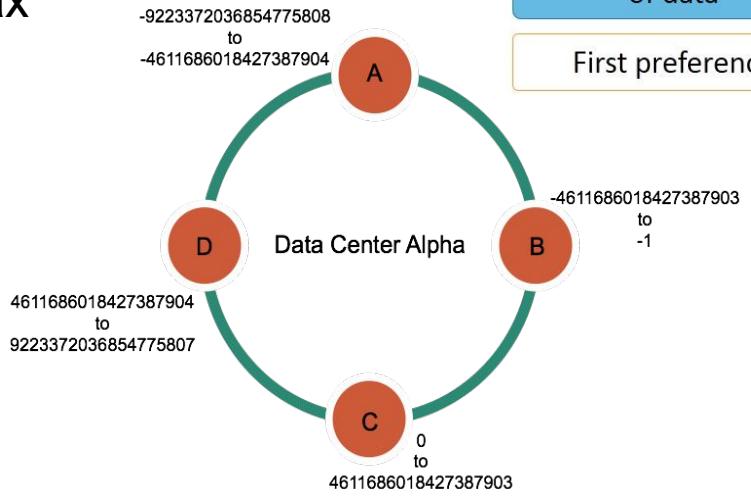


Устройство Cassandra

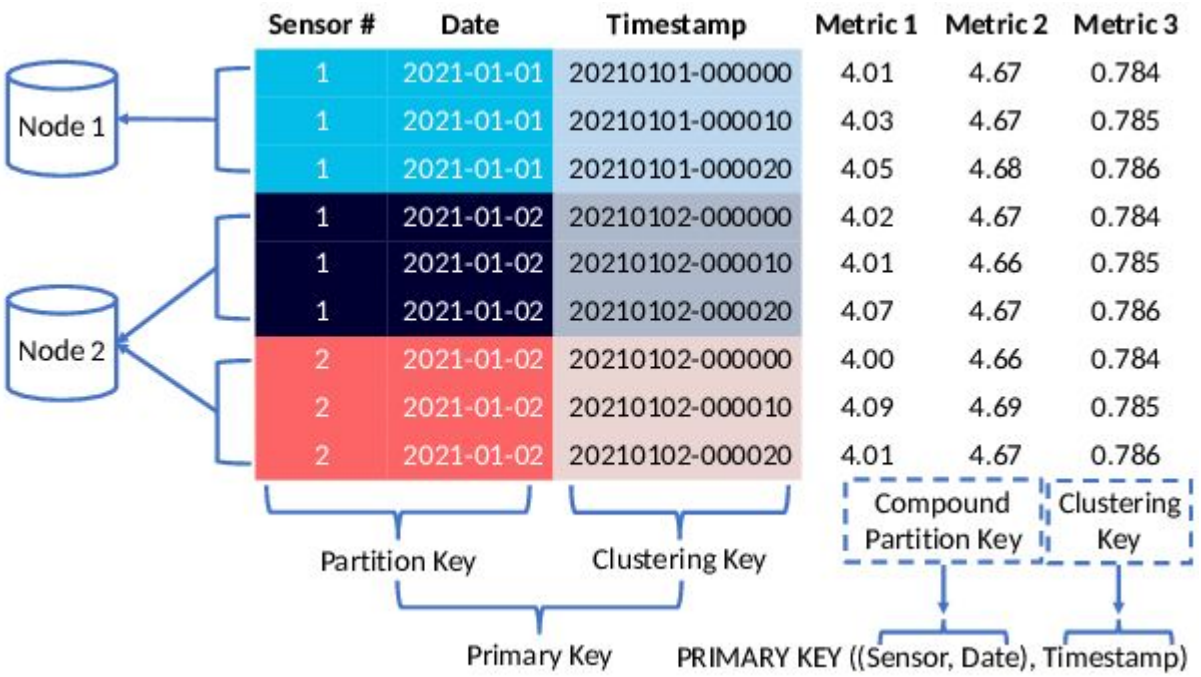
Элементы:

- Ring model (кольцевая модель) распределения данных
- Token range - блок токенов. На узле размещается несколько диапазонов токенов
- Реплики - копии строки на других узлах для резервирования





Устройство Cassandra



<u>Партицирование</u>:

- Primary Key - Первичный ключ строки может быть композитным (compound primary key)
- (Compound) Композитный ключ состоит из
  - Partition key (партиционный ключ)
    - имеет токен,
    - алгоритм hash-таблицы,
    - несортируемый,
    - используется для вычисления токена
  - Clustering key (кластерный ключ)
    - **колонки внутри одной партиции**,
    - сортируются.

Необходимо минимизировать число кластерных ключей - несовместим с большими данными

Все запросы по clustering key идут на ОДИН узел

Устройство Cassandra

Партицирование:

- Primary Key - Первичный ключ строки может быть композитным (compound primary key)
- (Compound) Композитный ключ состоит из
  - Partition key (партиционный ключ)
    - имеет токен,
    - алгоритм hash-таблицы,
    - несортируемый,
    - используется для вычисления токена
  - Clustering key (кластерный ключ)
    - **колонки внутри одной партиции**,
    - сортируются.

```
cqlsh> CREATE TABLE cycling.rank_by_year_and_name (
race_year int,
race_name text,
cyclist_name text,
rank int,
PRIMARY KEY ((race_year, race_name), rank)
);
```

race_year	race_name	rank	cyclist_name
2014	4th Tour of Beijing	1	Phillippe GILBERT
2014	4th Tour of Beijing	2	Daniel MARTIN
2014	4th Tour of Beijing	3	Johan Esteban CHAVES
2015	Giro d'Italia - Stage 11 - Forli > Imola	1	Ilnur ZAKARIN
2015	Giro d'Italia - Stage 11 - Forli > Imola	2	Carlos BETANCUR
2015	Tour of Japan - Stage 4 - Minami > Shinshu	1	Benjamin PRADES
2015	Tour of Japan - Stage 4 - Minami > Shinshu	2	Adam PHELAN
2015	Tour of Japan - Stage 4 - Minami > Shinshu	3	Thomas LEBAS

Необходимо минимизировать число кластерных ключей - несовместим с большими данными

Все запросы по clustering key идут на ОДИН узел

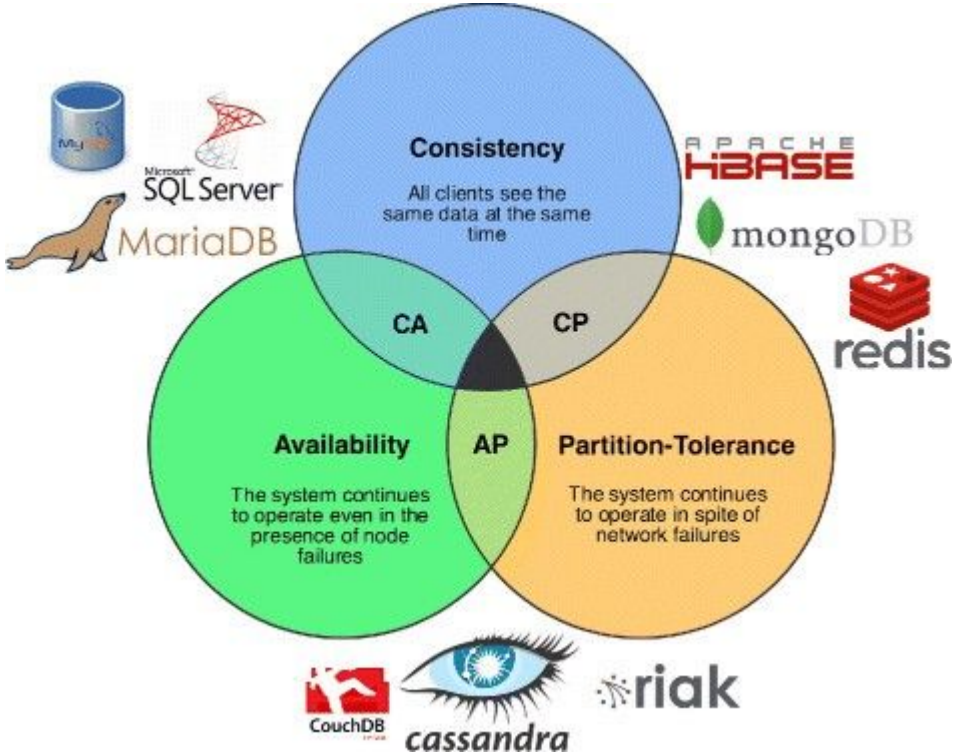
CAP-теорема

CAP:

consistency — во всех узлах в один момент времени данные не противоречат друг другу;
availability — запрос завершается корректно, но без гарантии совпадения ответов всех узлов;
partition tolerance — расщепление системы на несколько секций не приводит к некорректности отклика от каждой из секций.

Базы данных:

SQL - CA
Cassandra позволяет работать во всех режимах в зависимости от уровня консистентности: AP, CP, CA



Уровни консистентности

Сведения:

Определяют строгость проверки данных на согласованность
Вызывается ошибка при неудовлетворении согласованности данных в базе условиям запроса
Меняют тип работы БД: ONE/ANY - AP, ALL - CP, QUORUM - CA

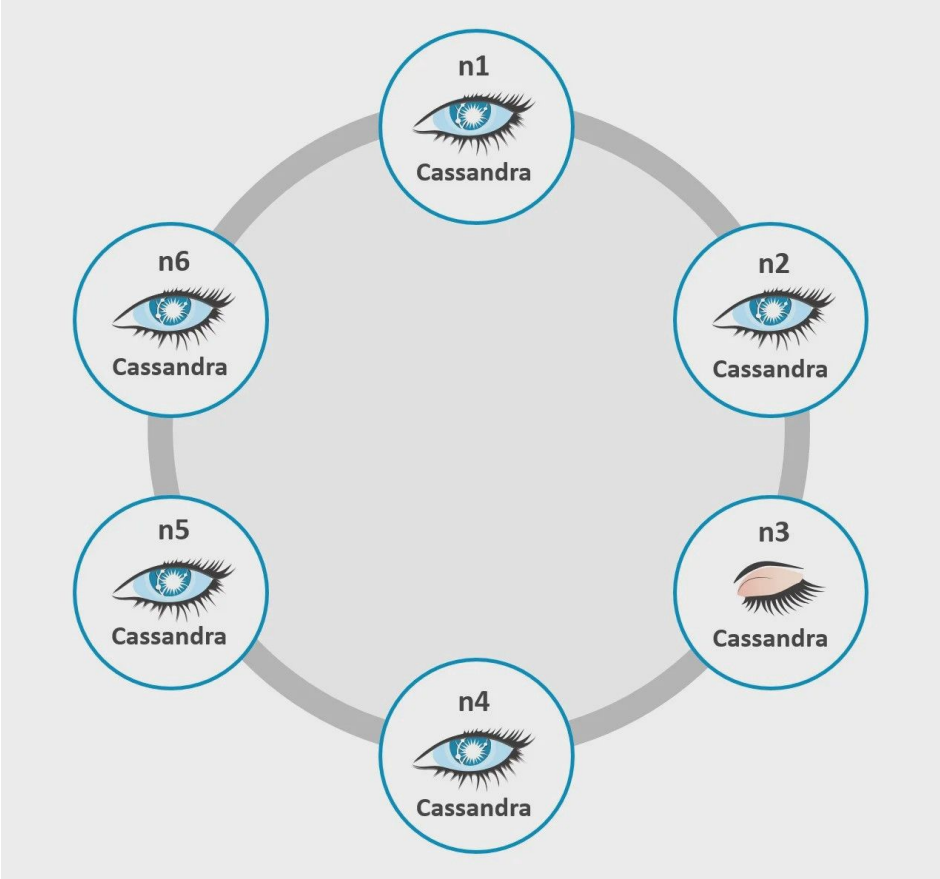
Базы данных:

ANY - не дожидаемся подтверждения завершения операции
ONE - операция подтверждена как минимум одним узлом
QUORUM - ⅔, ⅗, ¾, узла подтвердили выполнение операции
ALL - все узлы подтвердили выполнение операции

Level	Description	Usage
ALL	A write must be written to the commit log and memtable on all replica nodes in the cluster for that partition.	Provides the highest consistency and the lowest availability of any other level.
EACH_QUORUM	Strong consistency. A write must be written to the commit log and memtable on a quorum of replica nodes in each datacenter.	Used in multiple datacenter clusters to strictly maintain consistency at the same level in each datacenter. For example, choose this level if you want a write to fail when a datacenter is down and the QUORUM cannot be reached on that datacenter.
QUORUM	A write must be written to the commit log and memtable on a quorum of replica nodes across *all* datacenters.	Used in either single or multiple datacenter clusters to maintain strong consistency across the cluster. Use if you can tolerate some level of failure.
LOCAL_QUORUM	Strong consistency. A write must be written to the commit log and memtable on a quorum of replica nodes in the same datacenter as the coordinator. Avoids latency of inter-datacenter communication.	Used in multiple datacenter clusters with a rack-aware replica placement strategy, such as NetworkTopologyStrategy, and a properly configured snitch. Use to maintain consistency locally (within the single datacenter). Can be used with SimpleStrategy.
ONE	A write must be written to the commit log and memtable of at least one replica node.	Satisfies the needs of most users because consistency requirements are not stringent.
TWO	A write must be written to the commit log and memtable of at least two replica nodes.	Similar to ONE.
THREE	A write must be written to the commit log and memtable of at least three replica nodes.	Similar to TWO.
LOCAL_ONE	A write must be sent to, and successfully acknowledged by, at least one replica node in the local datacenter.	In a multiple datacenter clusters, a consistency level of ONE is often desirable, but cross-DC traffic is not. LOCAL_ONE accomplishes this. For security and quality reasons, you can use this consistency level in an offline datacenter to prevent automatic connection to online nodes in other datacenters if an offline node goes down.
ANY	A write must be written to at least one node. If all replica nodes for the given partition key are down, the write can still succeed after a hinted handoff has been written. If all replica nodes are down at write time, an ANY write is not readable until the replica nodes for that partition have recovered.	Provides low latency and a guarantee that a write never fails. Delivers the lowest consistency and highest availability.

Особенности программирования

1) Батчи - ТОЛЬКО для атомарных операций. В большинстве случаев использовать нельзя
2) Операции только асинхронные
3) Ожидание выполнения не по ~~Future.get~~, а по CompletionStage.await()
4) Проблема флудинга
   1) вызывает ошибки записи
   2) переполнение памяти на сервере
   3) выход сервера из строя
5) Меры против флудинга
   1) Запросы через очередь: не более N одновременных запросов (размер очереди)
   2) Регулирование размера очереди: число N увеличивается при снижении числа отказов и уменьшается при росте частоты ошибок
   3) Контроль размера кластера



Создание Репозитория

Подключение зависимостей

```
implementation("com.datastax.oss:java-driver-core: $cassandraDriverVersion ")
```

основная библиотека драйвера

```
implementation("com.datastax.oss:java-driver-query-builder: $cassandraDriverVersion ")
```

Query builder: позволяет конструировать CQL-запросы программно

```
kapt("com.datastax.oss:java-driver-mapper-processor: $cassandraDriverVersion ")
implementation("com.datastax.oss:java-driver-mapper-runtime: $cassandraDriverVersion ")
```

Object mapper: генерация шаблона для выполнения запросов и преобразования результатов в собственные классы

```
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-jdk8:1.3.8" )
```

Позволяет работать с CompletableStage

Entity

@Entity - таблица или пользовательский тип, аргументом может быть keyspace по умолчанию.

@PartitionKey - партиционный ключ, аргументом может быть позиция в составном партиционном ключе.

@CqlName - имя поля или класса в таблице

@PropertyStrategy - стратегия создания геттеров и сеттеров, для мутабельных полей указывается аргумент mutable = true

```
@Entity
data class ProductDto(
    @PartitionKey()
    @CqlName(COLUMN_ID)
    val id: String? = null,
    @CqlName(COLUMN_NAME)
    val name: String? = null,
    @CqlName(COLUMN_PRODUCER)
    val producer: ProducerDto? = null,
    @CqlName(COLUMN_PRICE)
    val price: Double? = null,
    @CqlName(COLUMN_DESCRIPTION)
    val description: String? = null,
    @CqlName(COLUMN_CREATED)
    val created: LocalDate? = null,
    @CqlName(COLUMN_LAST_WATCH)
    val lastWatch: String? = null,
)
```

DAO

@Dao - определяет набор методов запросов, связанных с dto.
@Select, @Insert, @Update, @Delete - определяют тип запроса.
@Query - пользовательский запрос, например:

```
@Query("SELECT count(*) FROM sensor_readings WHERE id = :id")

fun countById(int id): Long
```

@StatementAttributes - набор параметров запроса, например, можно устанавливать уровень консистентности или таймаут на ожидание ответа
@QueryProvider - генерация запроса программно

```
@Dao
interface ProductDao {

    @Select
    fun getAsync(id: String): ListenableFuture<ProductDto>

    @Select
    @StatementAttributes(consistencyLevel = "QUORUM")
    fun list(): ListenableFuture<Collection<ProductDto>>

    @Insert
    @StatementAttributes(consistencyLevel = "ONE")
    fun  saveAsync(product: ProductDto): ListenableFuture<Unit>
}
```

Mapper

Используется для генерации boilerplate кода выполнения запросов

@Mapper - для создания билдера используется kapt (annotation processor). Билдер, по умолчанию, формируется как <interface name>Builder.
@DaoFactory – помечает метод для создания DAO.
@DaoKeyspace - передает keyspace в DAO (необязательный параметр).
@DaoTable - передает таблицу в DAO (необязательный параметр).

```
@Mapper
interface ProductMapper {

    @DaoFactory
    fun productDao(
        @DaoKeyspace keyspace: String,
        @DaoTable table: String
    ): ProductDao
}
```

Создание сессии

- Сессия создается с помощью билдера, где указываются точки подключения, датацентр, параметры авторизации и т.д.
- Для сессии можно подключить пользовательскую конфигурацию с помощью withConfigLoader.
- Сессия используется для выполнения запросов и для создания маппера.

```
private val session by lazy {
    val builder = CqlSession.builder()
        .addContactPoints(parseAddresses(hosts, port))
        .withLocalDatacenter( localDatacenter: "datacenter1")
        .withAuthCredentials(user, pass)
    builder.build().apply {  this: CqlSession
        createKeyspace() // создание кейспейса
    }
    builder.withKeyspace(keyspaceName)
        .withConfigLoader(config).build().apply {  this: CqlSe
        createTypeProducer() // регистрация udt
        createTable()
    }  ^lazy
}
```

Создание keyspace

Стратегии репликации:

- SimpleStrategy - используется один датацентр, все реплики внутри датацентра.
- NetworkTopologyStrategy - позволяет масштабировать keyspace на несколько датацентров
- *LocalStrategy - для внутренних целей Cassandra*

```
    private fun CqlSession.createKeyspace() {
        execute(
            SchemaBuilder.createKeyspace(keyspaceName)
                .ifNotExists()
                .withSimpleStrategy(replicationFactor)
                .build()
        )
//        session.execute("""
//            CREATE KEYSPACE IF NOT EXISTS $keyspace WITH REPLICATION = {
//                'class' : 'SimpleStrategy',
//                'replication_factor' : $replicationFactor
//            }
//        """.trimIndent())
//        println(initSession.keyspace)
    }
```

Регистрация собственных типов

- Для собственного типа создается data class, помеченный аннотацией @Entity.
- С помощью SchemaBuilder создается тип с указанием всех полей созданного entity.
- Для использования в таблице тип помечается как SchemaBuilder.udt(TYPE_NAME, true)

```
@Entity
data class ProducerDto(
    @CqlName(COLUMN_NAME)
    val name: String? = null,
    @CqlName(COLUMN_SITE)
    val site: String? = null,
)

private fun CqlSession.createTypeProducer() {
    execute(
        SchemaBuilder.createType(TYPE_NAME)
        .ifNotExists()
        .withField(
            ProducerDto.COLUMN_NAME,
            DataTypes.TEXT)
        .withField(COLUMN_SITE, DataTypes.TEXT)
        .build()
    )
}
```

Создание таблицы

- Для создания таблицы в билдере указывается партиционный ключ (или несколько), колонки и кластерный ключ (опционально).
- Названия и тип колонок должны соответствовать таковым в entity.
- Прежде чем использовать собственный тип в таблице, нужно его зарегистрировать.

```
private fun CqlSession.createTable() {
    execute(
        SchemaBuilder.createTable(TABLE_NAME)
        .ifNotExists()
        .withPartitionKey(COLUMN_ID, DataTypes.TEXT)
        .withColumn(COLUMN_NAME, DataTypes.TEXT)
        .withColumn(COLUMN_PRODUCER, SchemaBuilder.udt(TYPE_NAME, frozen: true))
        .withColumn(COLUMN_PRICE, DataTypes.DOUBLE)
        .withColumn(COLUMN_DESCRIPTION, DataTypes.TEXT)
        .withColumn(COLUMN_CREATED, DataTypes.DATE)
        .withColumn(COLUMN_LAST_WATCH, DataTypes.TEXT)
        .build()
    )
}
```

LIVE

Рефлексия

Основные тезисы

1.	Cassandra - распределенная БД с высокой масштабируемостью и надежностью
2.	Надежность за счет репликации, масштабируемость из-за отсутствия мастера
3.	Данные распределены по партиционным ключам, кластерные ключи на одном узле
4.	При запросах нужно делать много мелких асинхронных запросов на запись, no batches
5.	Для тестирования применяется TestContainers со случайными портами

Задание

1) Как устроена Cassandra?
2) Какие особенности нужно учитывать при разработке с Cassandra?
3) Как происходит тестирование Cassandra с помощью TestContainers?

Домашнее задание

1. Выполнить InMemory реализацию репозитария
2. Сделать автоматические тесты для репозитория
3. Создать персистентное хранилище для выпускного проекта
4. Интегрировать хранилище в выпускной проект

Срок: 17.10.2025

Рефлексия

По какому вопросу захотелось глубже изучить информацию?

Понимаете ли вы, как применять на практике то, что узнали на вебинаре. Если да, то как?

Цели вебинара

Проверка достижения целей

1.	Узнать об особенностях устройства Cassandra и ее применении для хранения больших данных
2.	Создать репозитарий с использованием Cassandra
3.	Интегрировать репозитарий в приложении совместно с тестовыми БД

Список материалов для изучения

- https://kotlinlang.org/
- https://gradle.org/
- https://github.com/otuskotlin/202505-ok-marketplace
- https://habr.com/ru/post/155115
- https://docs.datastax.com/en/developer/java-driver/4.17/manual
- https://cassandra.apache.org/doc/latest/cassandra/cql/index.html
- https://github.com/DataStax-Examples/object-mapper-jvm/tree/master/kotlin

Следующий вебинар

14.10.2025

Модуль 7: Работа с базами данных
Репозитарий с графовой базой данных

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары